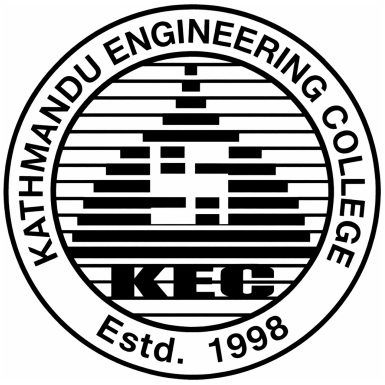
Tribhuvan University

Institute of Engineering

Kathmandu Engineering College

Kalimati, Kathmandu



Project Report on

SPARK: Explainable Edge AI for Kinetic Pattern Recognition and Distress Signaling

SUBMITTED BY:

Aaradhya Dev Tamrakar (79001)

Rupesh Kadel (79034)

Sankalpa Lamsal (79039)

Sonia Thapa (79043)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS, COMMUNICATION AND INFORMATION ENGINEERING

July 2026

Acknowledgement

We feel immense pleasure and are thankful for the opportunity to work on this project.

We would like to sincerely thank our supervisor, **Er. Dipen Manandhar**, for his unlimited encouragement and timely support and guidance in helping us with the different aspects of the project. We owe our profound gratitude to our project coordinator, **Er. Sujan Shrestha**, who took a keen interest in our project and guided us all along, providing the necessary information for developing a robust system.

We would like to thank **Er. Suramya Sharma Dahal**, Head of the Department of Electronics, Communication, and Information Engineering, for providing us with the opportunity to perform this project.

We are thankful to and fortunate enough to get constant encouragement, support, constructive criticism, and guidance from all teaching staff of the Department of Electronics, Communication, and Information Engineering.

Abstract

SPARK is an open-hardware wearable fall detection system for Nepal's growing elderly care gap. An ESP32-S3 and MPU6050 IMU node, worn on the wrist (top-of-wrist/dorsal placement, Velcro closure), runs a two-layer on-device pipeline a threshold pre-filter (Layer 1) gating a quantized INT8 TFLite Micro 1D CNN (Layer 2) with no cloud dependency. Confirmed falls are transmitted over BLE to a laptop gateway, which computes SHAP feature attribution per event so classifications are auditable rather than a black-box verdict, and auto-generates a PDF clinical incident report. A companion phone display client provides caregivers a read-only view of completed reports without requiring the laptop to be physically present.

The project contributes a Nepal-context wearable IMU fall dataset, a configurable sensitivity strategy (High / Standard / Sport), and an open-source reference design for low-cost elderly fall monitoring across South Asian care settings.

Keywords: BLE, edge AI, elderly care, ESP32-S3, fall detection, IMU, Nepal, SHAP explainability, TFLite Micro, wearable sensor.

Table of Contents

List of Abbreviations

ADL	Activities of Daily Living
API	Application Programming Interface
BEI	Bachelor of Electronics, Communication and Information Engineering
CNN	Convolutional Neural Network
ESP32	Espressif Systems ESP32 DevKit V1
IMU	Inertial Measurement Unit
INT8	8-bit Integer Quantization
IOE	Institute of Engineering
ISO	International Organization for Standardization
KEC	Kathmandu Engineering College
MQTT	Message Queuing Telemetry Transport
MPU6050	InvenSense MPU-6050 6-axis IMU
RPi	Raspberry Pi
SHAP	SHapley Additive exPlanations
SPARK	Signal Pattern Analysis & Real-time Kinetics
TFLite	TensorFlow Lite
WHO	World Health Organization

List of Figures

List of Tables

Chapter 1: Introduction

1.1 Background

Falls are a leading cause of injury-related mortality and morbidity among adults aged 65 and above worldwide. The WHO Global Report on Falls Prevention in Older Age (2007) identifies falls as the second leading cause of unintentional injury death globally, with approximately 684,000 fatal falls occurring each year [1]. Non-fatal falls cause serious injuries including hip fractures, traumatic brain injuries, and long-term disability, placing a heavy burden on healthcare systems and caregivers.

Nepal is undergoing a rapid demographic shift. The Central Bureau of Statistics projects that Nepal's population aged 65 and above will double by 2035 [2], driven by improving life expectancy and urbanization. Despite this growing demographic pressure, no domestic fall detection product exists in the Nepali market. Commercial solutions such as the Apple Watch and medical alert pendants are cost-prohibitive — a single Apple Watch SE (2nd generation) retails above NPR 30,000 — and require proprietary ecosystems unavailable to most Nepali families. A local BLE link between the wearable and a household laptop gateway requires no internet connectivity at all, making SPARK deployable in care settings regardless of WiFi/ISP reliability.

Academic fall detection systems published in the literature typically rely on one of two approaches: threshold-only detection on raw accelerometer data, which produces unacceptably high false-positive rates on activities of daily living such as sit-to-stand transitions and stair climbing; or cloud-dependent machine learning inference, which introduces unacceptable network dependencies for a safety-critical application. Neither approach has been validated on a Nepal-context dataset, nor do any of the published open-source systems provide per-event classifier explainability for care staff.

SPARK addresses these gaps with a two-layer gated edge AI architecture running entirely on an ESP32 microcontroller, a SHAP-based explainability pipeline at the gateway, and a self-collected Nepal-context IMU dataset collected at KEC.

1.2 Problem Statement

Falls are the leading cause of injury-related mortality among adults over 65 globally [1]. Nepal's elderly population is projected to double by 2035 [2], yet no affordable domestic fall detection solution exists. Commercial fall detection systems (Apple Watch, Samsung

Galaxy Watch, Withings ScanWatch) are cost-prohibitive for Nepal's eldercare context, require proprietary ecosystems, and operate as closed black-box systems — returning a binary fall verdict with no classifier explanation. Published academic systems use either threshold-only detection (unacceptably high false-positive rate) or cloud-dependent machine learning inference (network dependency unacceptable for safety-critical applications in Nepal's connectivity environment). No published open-hardware system demonstrates:

1. A two-layer gated architecture with on-device TFLite Micro inference.
2. SHAP-based per-event explainability surfaced to caregivers.
3. Adaptive sensitivity profiles.
4. Clinical-format automated incident reporting validated on a Nepal-context IMU dataset.

1.3 Objectives

This project focuses on building a wearable fall detection system that combines hardware, software, and edge AI to automate real-time monitoring and caregiver alerting. The scope includes the following key objectives:

1. To construct the hardware structure of a wearable edge computing node.
2. To integrate a two-layer gated AI architecture for cloud-independent, real-time fall classification.
3. To implement SHAP-based explainability techniques to log and render transparent feature attribution per event.

1.4 Scope of the Project

The scope of our project includes:

- **On-Device Data Acquisition:** We sample and filter 6-axis accelerometer and gyroscope data directly on the wearable node at a continuous 200 Hz frequency.
- **Dual-Layer Gated Inference:** Our system uses a fast mathematical threshold check to wake up an on-device, quantized 1D CNN classifier only when a potential fall signature is detected.
- **BLE Event Transmission:** Confirmed fall events are serialized as JSON and transmitted over Bluetooth Low Energy to a laptop gateway, requiring no WiFi or cloud

connectivity.

- **SHAP Explainability Pipeline:** The system calculates Shapley additive feature values at the gateway to show caregivers exactly which sensor movements triggered the alarm.
- **Read-Only Phone Display Client:** A companion phone view reads completed reports from the laptop gateway over the local network, letting caregivers check results without the laptop present.
- **Automated Clinical Reporting:** The pipeline automatically packages peak movement data and model confidence ratings into formal PDF incident summary files.

1.5 Application of the Project

1. Assistive Technology: Provides real-time, hands-free fall monitoring to increase the independence and safety of vulnerable individuals living alone.
2. Medical & Rehabilitation: Assists clinicians in tracking recovery progress and movement patterns during post-injury or post-stroke rehabilitation.
3. Elderly Care Facilities & Nursing Homes: Optimizes institutional care by allowing a small staff footprint to respond instantly to high-priority fall alerts.
4. Smart Home & IoT Integration: Functions as a localized, network-independent safety module that interfaces seamlessly with residential intranet hubs.
5. Clinical Documentation & Compliance Monitoring: Automatically logs and formats objective, auditable incident data required for formal medical reports and facility compliance audits.
6. Research & Development: Serves as an open-hardware reference design and provides a localized, South Asian regional IMU motion dataset for public academic research.

Chapter 2: Literature Review

Fall detection using wearable inertial sensors has been an active research area for over two decades. The literature can be broadly categorized by detection approach: threshold-based methods, machine learning classifiers, and more recent deep learning architectures.

Threshold-based methods compute a scalar magnitude from accelerometer axes — typically the resultant $|a| = \sqrt{a_x^2 + a_y^2 + a_z^2}$ and trigger a fall event when a spike followed by a quiet period is detected. Kangas et al. [3] compared multiple threshold algorithms on hip and wrist IMU data and reported sensitivity values of 73–95% depending on placement, but with false-positive rates exceeding 30% on vigorous ADLs. Bourke and Lyons [4] proposed a dual-axis threshold method achieving 96% sensitivity, though specificity on sit-to-stand transitions remained problematic. The fundamental limitation of threshold-only approaches is that any rapid motion — vigorous exercise, dropping an object, stumbling without falling — can trigger the threshold, requiring either a very high threshold (missing genuine falls) or a low threshold (excessive false positives).

Machine learning classifiers applied to hand-crafted time-domain and frequency-domain features from IMU windows have addressed some of these limitations. Sucerquia et al. published the SisFall dataset [5], a widely used benchmark containing 15 fall types and 19 ADL types from 38 subjects, enabling standardized evaluation of feature-based classifiers. Support vector machines, decision trees, and random forests trained on SisFall features have achieved sensitivity above 90%, but these approaches typically run on a connected laptop or server, not on the wearable hardware itself.

Deep learning on IMU time-series has demonstrated superior accuracy without manual feature engineering. Nho et al. [7] trained a 1D CNN on the UCI MHEALTH dataset and SisFall combined, reporting sensitivity of 97.2% and specificity of 98.1% with a model that fits within 150 KB. However, the model ran on a Raspberry Pi 3 serving as the wearable compute unit, not on a resource-constrained microcontroller. Wang et al. [8] applied an LSTM architecture to continuous IMU streams, achieving comparable accuracy but with inference latency unsuitable for real-time detection on embedded hardware.

TFLite Micro on microcontrollers has enabled CNN deployment on devices with fewer than 320 KB of RAM. David et al. [9] demonstrated keyword spotting and gesture recognition using TFLite Micro on Cortex-M4 class hardware, establishing the feasibility of INT8 quantized CNN inference in this memory envelope. Kumar et al. [10] deployed a threshold-gated classifier on an Arduino Nano 33 BLE Sense, but did not combine on-device TFLite

Micro CNN inference with a structured two-layer gating architecture or provide any gateway explainability.

Explainability in health AI has been identified as a critical gap for clinical adoption. Lundberg and Lee [11] introduced SHAP (SHapley Additive exPlanations), a unified framework for model interpretability based on cooperative game theory, which provides consistent attribution of each input feature's contribution to a model output. While SHAP has been applied to ECG classification [12] and sepsis prediction in clinical ML, no published wearable fall detection system applies SHAP attribution at the per-event level and surfaces the result to caregivers in a clinical dashboard.

Gap summary: The existing literature leaves the following cluster of gaps unaddressed simultaneously: (a) on-device TFLite Micro CNN inference in a two-layer gated architecture on a sub-$5 microcontroller; (b) SHAP-based per-event explainability surfaced to caregivers; (c) adaptive sensitivity profiles without firmware reflash; and (d) a dataset collected from Nepali subjects on locally procurable hardware. SPARK addresses all four gaps in a single open-source system.

Chapter 3: Related Theory

3.1 Hardware Description

3.1.1 ESP32-S3

The ESP32-S3 (Espressif Systems, WROOM-1-CAM/N16R8 variant) is a dual-core Xtensa LX7 microcontroller operating at up to 240 MHz, with 512 KB SRAM, 16 MB flash, and integrated 802.11 b/g/n WiFi and Bluetooth 5 (LE). For SPARK, only the BLE subsystem is used for gateway transmission; WiFi is disabled to conserve power. The ESP32-S3 executes both the sensor acquisition ISR at 200 Hz and the two-layer fall detection pipeline including TFLite Micro CNN inference within the available SRAM and flash envelope. INT8 quantization reduces the 1D CNN model footprint to approximately 80–120 KB of flash, leaving sufficient headroom for the Arduino SDK, BLE stack, and ring-buffer firmware.

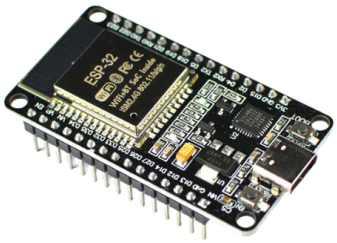


Figure 3.1: ESP32-S3

3.1.2 MPU6050 6-Axis IMU

The InvenSense MPU-6050 integrates a 3-axis MEMS accelerometer (±2/±4/±8/±16 g selectable) and a 3-axis MEMS gyroscope (±250/±500/±1000/±2000°/s selectable) on a single die, communicating over I^2C at up to 400 kHz. For SPARK, the accelerometer is configured at ±8 g full-scale (captures the free-fall and impact spike of a genuine fall without clipping) and the gyroscope at ±500°/s. The built-in Digital Motion Processor (DMP) is bypassed in favour of raw register reads at 200 Hz, enabling direct integration with the complementary filter and sliding window buffer.

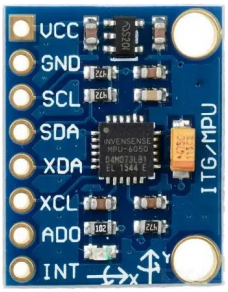


Figure 3.2: MPU6050 6-Axis IMU

3.1.3 Laptop Gateway

A laptop (Acer Swift Go 16, already owned by the team) serves as the sole SPARK gateway: sole BLE receiver and sole SHAP/PDF pipeline compute. It receives the JSON event payload over BLE from the wearable node, runs the SHAP attribution pipeline locally against the trained CNN, and auto-generates a one-page clinical PDF incident report — all without a cloud round-trip or persistent database. No Raspberry Pi or dedicated single-board computer is required; the laptop is the gateway for both development and the March 2027 demonstration.

3.1.4 Phone Display Client

A user's own phone (no dedicated purchase) acts as a read-only Layer 3 client. It has no direct BLE link to the wearable and performs no SHAP computation or PDF generation on-device; it reads a completed report from the laptop gateway over the local network, letting a caregiver or patient view results without the laptop physically present.

3.1.5 Power System (Wearable Node)

The wearable node is powered by a single 3.7 V nominal 1100 mAh Li-ion/LiPo cell, paired with a TP4056-class charge/protection circuit (USB-C input, 1 A charge current) required unless the sourced pack is already protected. No separate LDO regulator or current/voltage telemetry sensor is used in this design — the ESP32-S3 devboard and MPU6050 breakout both carry their own onboard regulation, and no power-rail instrumentation exists to log real-time draw. At an estimated 110–140 mA active current draw (BLE + 200 Hz IMU polling), the 1100 mAh cell is projected to provide approximately 8–10 hours of continuous monitoring; this is an estimate from published ESP32-S3 active-draw figures, not a measured value, pending on-hardware profiling.

3.1.5.1 Li-ion/LiPo Cell

A single 3.7 V nominal 1100 mAh Li-ion/LiPo pouch cell that serves as the main power source for the wearable node. At an estimated active current draw of 110–140 mA, it is projected to provide approximately 8–10 hours of continuous, uninterrupted fall monitoring, pending on-hardware verification.

Figure 3.3: Li-ion/LiPo Cell

3.1.5.2 Charge/Protection Circuit

A single-cell constant-current/constant-voltage (CC/CV) linear charge controller module (TP4056-class) with USB-C input. It regulates a 1 A charging current to safely recharge the cell and provides safety cutoffs (overcharge protection at 4.2 V and over-discharge protection at 2.5 V). Required unless the sourced battery pack is already protected.

Figure 3.4: Charge/Protection Circuit

3.1.6 Mechanical Mounting and Support Components

Beyond the core compute, sensing, and power modules, the wearable node relies on a set of mechanical mounting and low-level support components for physical attachment and circuit integration.

3.1.6.1 Velcro Wrist Strap

An adjustable Velcro strap used to secure the wearable enclosure to the wrist (top-of-wrist/dorsal placement, locked design), allowing the accelerometer and gyroscope to reliably capture body motion during a fall event. This strap is a separate, additional-retention component distinct from the enclosure's own built-in Velcro closure tab (part of the 3D-printed shell, no separate purchase).

Figure 3.5: Velcro Wrist Strap

3.1.6.2 USB-C Cables

Standard USB-C cables used to power and program the ESP32-S3 boards and to charge the wearable's battery via the onboard charge/protection circuit during bench testing.

Figure 3.6: USB-C Cable

3.1.6.3 Passive Components

Assorted resistors and capacitors used on the wearable node breadboard for signal conditioning, decoupling, and pull-up/pull-down configuration around the ESP32-S3 and MPU6050 circuitry.

Figure 3.7: Resistor

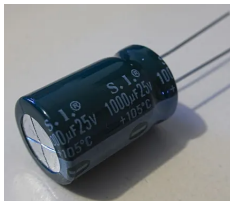


Figure 3.8: Capacitor

3.2 Software Description

3.2.1 Two-Layer Detection Architecture

The detection pipeline implements a gated dual-layer structure. Layer 1 is a deterministic threshold pre-filter executed in the ESP32-S3 ISR at 200 Hz. It computes the resultant acceleration magnitude $|a| = \sqrt{a_x^2 + a_y^2 + a_z^2}$. A `PRE_FALL` flag is raised when $|a|$ exceeds 2.5 g for more than 300 ms; the queue is cleared if no new peak acceleration occurs within a 10 s timeout. These thresholds are currently fixed design values, pending empirical tuning against real fall simulations (WP 1.0/2.0). Layer 1 incurs no inference cost, reducing false-positive load on the CNN and conserving power by skipping inference on clear non-falls.

When `PRE_FALL` is raised, Layer 2 is invoked: the gate-triggered window of $(a_x, a_y, a_z, \omega_x, \omega_y, \omega_z)$ at 200 Hz is assembled, normalized using StandardScaler parameters baked into the firmware at training time, and fed to the TFLite Micro 1D CNN. The CNN produces a fall probability score $P(\text{FALL}) \in [0, 1]$. If $P(\text{FALL}) > 0.85$, a `CONFIRMED_FALL` event is serialized as JSON — device ID, firmware version, timestamp, confidence, and per-channel peak features, per the locked wire format — and transmitted to the laptop gateway over BLE.

3.2.2 1D CNN Architecture

The 1D CNN consists of: Conv1D(32 filters, kernel 5) → ReLU → MaxPool1D(2) → Conv1D(64 filters, kernel 3) → ReLU → GlobalAveragePooling1D → Dense(32, ReLU) → Dense(2, Softmax). The model is trained in TensorFlow/Keras on the SisFall dataset [5] as the primary source, with self-collected Nepal-context data (protocol TBD) as a secondary fine-tuning set. INT8 post-training quantization via the TFLite Converter produces a C array for inclusion in the PlatformIO build.

3.2.3 SHAP Explainability Pipeline

SHAP (SHapley Additive exPlanations) [11] assigns to each input feature i an attribution value ϕ_i satisfying local accuracy ($f(x) = \phi_0 + \sum_i \phi_i$), missingness, and consistency axioms. For each `CONFIRMED_FALL` event, the laptop gateway computes SHAP values for the six IMU channel features (peak values of $a_x, a_y, a_z, \omega_x, \omega_y, \omega_z$ received over BLE) using the `shap` Python library against the trained CNN output. The top contributing feature (e.g., `az_peak` with 42% of fall confidence) and the full SHAP vector are written directly into the auto-generated PDF incident report — computed and rendered locally on the laptop, with no cloud round-trip and no persistent database. This provides care staff with an auditable rationale for every alert — a capability absent from all commercial and published academic fall detection systems.

3.2.4 Gateway Software Architecture

The gateway implements three software design patterns documented in the thesis Chapter 3 (OOSE alignment):

Factory Pattern decouples the detection node abstraction from its concrete implementations. A `FallDetector` abstract base class handles BLE transmission, event serialization, and power management; `ThresholdDetector` (Layer 1) and `CNNDetector` (Layer 2) are concrete subclasses.

Observer Pattern implements the report-generation pipeline. The BLE receiver is the Subject; the SHAP attribution module and PDF report generator are Observers subscribed to `CONFIRMED_FALL` events. The read-only phone display client is not itself an Observer — it reads the completed report from the laptop over the local network once generation finishes. Adding a new local output (e.g., a CSV export) requires adding a new Observer without modifying the gateway core.

Strategy Pattern implements the configurable sensitivity profiles. A `FallDetectionStrategy` interface is implemented by three concrete strategies: `HighSensitivity` (elderly, bedridden), `Standard` (default), and `SportMode` (active ADL). Threshold parameters are loaded from the user profile at gateway startup; no firmware reflash is required to switch profiles.

Chapter 4: Feasibility Study

4.1 Technical Feasibility

SPARK is technically feasible because every component of the system has been individually demonstrated in the published literature, and the complete hardware stack is either in-hand or procurable locally in Kathmandu within one day.

On-device CNN inference on ESP32-S3. TensorFlow Lite Micro has been demonstrated on Cortex-M4 class hardware with flash footprints below 250 KB [9]. The ESP32-S3 dual-core Xtensa LX7 processor (240 MHz, 512 KB SRAM, 16 MB flash) provides a larger compute envelope than the Cortex-M4 targets evaluated by David et al. INT8 post-training quantization reduces the SPARK 1D CNN to approximately 80–120 KB of flash, leaving sufficient headroom for the Arduino SDK, BLE stack, and ring-buffer firmware. The sliding window buffer and Layer 1 threshold logic together add fewer than 4 KB of RAM and run at 200 Hz well within the ISR budget.

Two-layer gated detection architecture. Layer 1 threshold logic is a scalar comparison on the resultant acceleration magnitude $|a|$ against a fixed threshold and duration. Computation completes in under 1 ms on the ESP32-S3 at 200 Hz. Layer 2 CNN inference, when gated by Layer 1, runs only on candidate fall windows (typical rate: fewer than 5 per hour of normal ADL), keeping average power overhead negligible.

BLE transport. The ESP32-S3's integrated Bluetooth 5 (LE) radio delivers the locked JSON event payload (device ID, timestamp, confidence, six peak features — well under a single BLE packet at typical negotiated MTU) directly to the laptop gateway, with no broker or intermediate network hop required. WiFi is disabled on the wearable node to conserve power; the laptop (Acer Swift Go 16, already owned by the team) is the sole BLE receiver for both development and the March 2027 demonstration, with no Raspberry Pi or dedicated single-board computer needed.

SHAP gateway pipeline. The `shap` Python library computes feature attributions for the 6-feature CNN output on the laptop per confirmed fall event, entirely locally with no cloud round-trip. Per-event SHAP computation is triggered only on confirmed fall events, not continuously, so gateway CPU load is minimal; the resulting attribution is written directly into the auto-generated PDF incident report.

Self-collected dataset. Controlled fall simulation on a crash mat is a standard data-collection protocol used by Sucerquia et al. [5] (38 subjects) and Kumar et al. [10]. Four team members

plus volunteers recording on-campus fall simulations at KEC can achieve the 500-fall-event / 2000-ADL-window target within four weeks. No ethical approval is required for healthy adult volunteers performing low-risk controlled falls in a supervised laboratory setting.

No unsolved technical dependencies. All software libraries (TFLite Micro, SHAP, PDF generation) are open source and actively maintained. No custom PCB fabrication, no import of restricted components, and no cloud service subscriptions are required.

4.2 Economic Feasibility

SPARK requires no imported components; all hardware is procurable locally from Himalayan Solutions, Giga Nepal, RR-papers, or Daraz (Kathmandu), with the enclosure 3D-printed in-house at KEC Makerspace. The gateway (laptop) and Layer 3 display client (phone) are both already owned by the team, at no additional cost. Table 4.1 summarises the estimated procurement cost for the wearable node BOM, covering three ESP32-S3 units (quantization testbed, deployment wearable, and spare) plus development consumables. Detailed component specifications are listed in Table 4.1.

Component	Qty	Unit (NPR)	Total (NPR)	Reference
ESP32-S3	3	1,800	5,400	Himalayan Solutions
MPU6050 IMU module (GY-251)	1	350	350	Himalayan Solutions
Li-ion/LiPo cell, 1100 mAh	1	550	550	Giga Nepal
Charge/protection circuit (TP4056-class)	1	90	90	baseline
Compression arm sleeve, thumb-hole	1	136	136	Daraz
Velcro wrist strap	2	500	1,000	RR-papers
Enclosure (3D-printed TPU, KEC Makerspace)	40 g (est.)	4/g	160	KEC Makerspace
USB-C cables	3	267	801	Daraz
Breadboard + jumper wires	2 sets	325	650	baseline
Resistors / capacitors assortment	1 lot	600	600	baseline
		Total	**9,737**	

Table 4.1: Component Cost Estimation (laptop gateway and phone display client already owned, NPR 0)

Chapter 5: Methodology

5.1 System Block Diagram

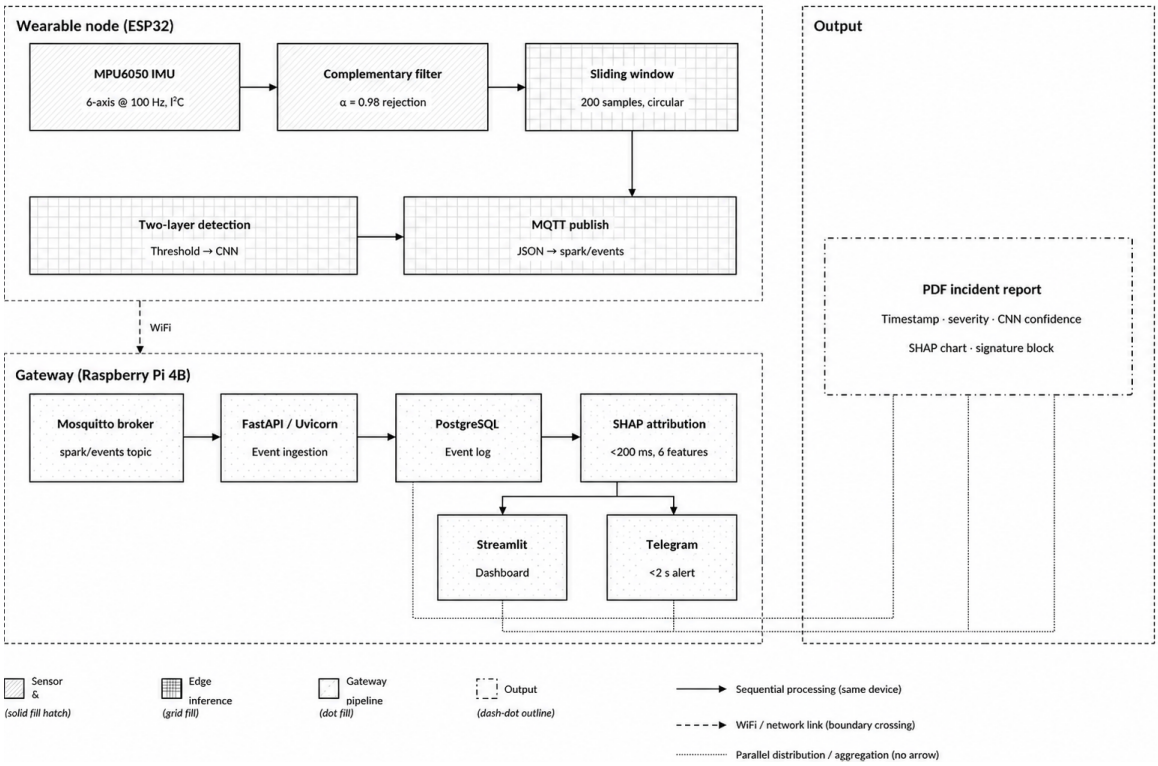


Figure 5.1: System Block Diagram

SPARK consists of two physical subsystems: a battery-powered wearable node and a laptop gateway, connected over BLE, plus a read-only phone display client on the local network. Figure 5.1 illustrates the top-level system architecture.

The wearable node (ESP32-S3 + MPU6050) acquires raw 6-axis IMU data at 200 Hz and runs the two-layer fall detection pipeline entirely on-device. On a confirmed fall, a JSON event payload is transmitted to the laptop gateway over BLE. The laptop ingests the event, computes SHAP attribution, and auto-generates a PDF clinical incident report — entirely locally, with no cloud round-trip or persistent database. A companion phone display client reads the completed report from the laptop over the local network. Figure 5.1 illustrates the end-to-end data and control flow across all pipeline stages.

Figure 5.1 is organized into three zones connected by two link types solid arrows for sequential on-device processing, and a dashed BLE boundary where the wearable node hands off to the gateway. Each block is described below.

Sensor zone (wearable node, solid fill hatch)

- **MPU6050 IMU** : reads all six raw accelerometer and gyroscope axes over I^2C at 200 Hz (Section 5.3.1).
- **Sliding window** : buffers the raw stream in a circular ring buffer, continuously overwritten until a candidate fall window is latched by Layer 1.

Edge inference zone (wearable node, grid fill)

- **Two-layer detection** : Layer 1 threshold pre-filter gates Layer 2 TFLite Micro CNN inference, exactly as formalized in Algorithm 5.4.
- **BLE transmit** : on `CONFIRMED_FALL`, serializes the event as JSON per the locked wire format and transmits it over Bluetooth Low Energy, crossing the BLE boundary to the laptop gateway.

Gateway zone (laptop, dot fill)

- **BLE receiver** : the sole receiver of `CONFIRMED_FALL` events from the wearable node, running locally on the laptop.
- **SHAP attribution** : computes per-feature attributions on the six peak IMU channels, identifying which channel most contributed to the fall classification.
- **PDF incident report** : auto-generates a one-page clinical report containing the SHAP attribution chart and a care-staff signature block, readable by the phone display client over the local network.

5.2 Hardware Implementation

5.2.1 Wearable Node Assembly

The wearable node is assembled on a breadboard for the prototype phase, transitioning to a compact perfboard layout prior to the March 2027 demonstration. The assembly sequence is:

1. Connect the MPU6050 to the ESP32-S3 via I^2C: SDA to GPIO 21, SCL to GPIO 22, operating at 400 kHz.
2. Power the MPU6050 from the ESP32-S3 3.3 V rail (onboard regulation; no separate LDO required).

3. Connect the 1100 mAh Li-ion/LiPo cell output to the TP4056-class charge/protection circuit (required unless the sourced pack is already protected), then to the ESP32-S3 devboard's battery input.

4. House the assembly in a 3D-printed TPU enclosure (KEC Makerspace) with a Velcro wrist strap, the enclosure's own built-in Velcro closure tab, and a flush-mounted USB-C charging port.

5.2.2 Power Budget Verification

No power-rail telemetry sensor is used in this design. Target metrics: active current draw estimated at 110–140 mA (BLE + 200 Hz IMU polling), providing an estimated 8–10 hours of continuous monitoring from the 1100 mAh cell. This is a published-figure estimate, not yet measured on hardware; on-hardware profiling and documentation is planned for the thesis Chapter 5 (Implementation).

5.3 Firmware Implementation (ESP32 PlatformIO)

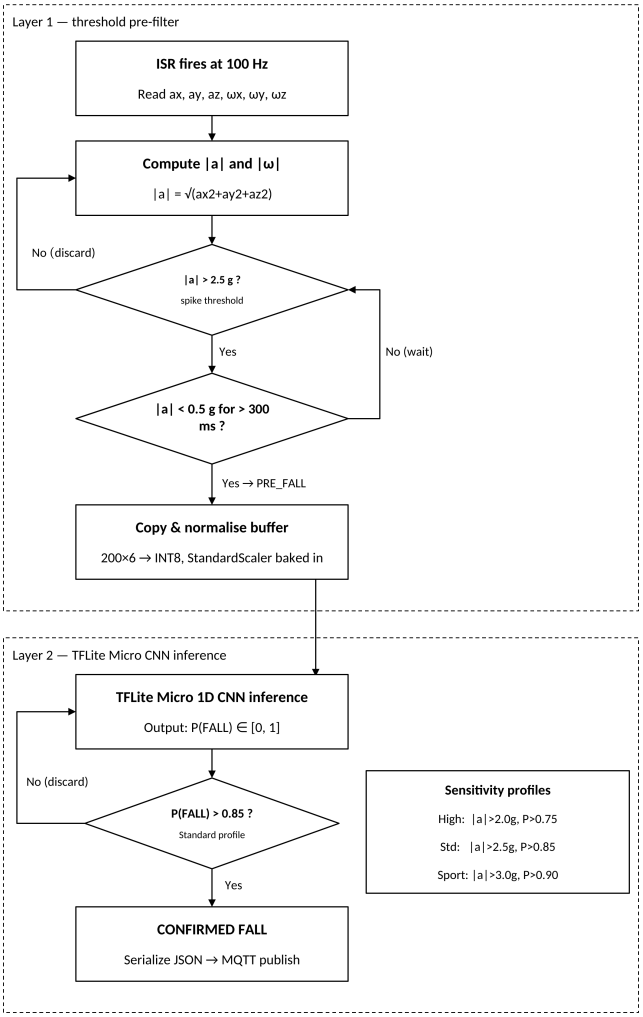


Figure 5.2: Two-layer fall detection flowchart: Layer 1 threshold pre-filter (top zone) gates Layer 2 TFLite Micro CNN inference (bottom zone). Sensitivity profile thresholds are shown in the callout.

5.3.1 IMU Acquisition and Filtering

The MPU6050 is configured via I^2C at startup: accelerometer full-scale ±8 g, gyroscope full-scale ±500°/s. A timer interrupt service routine (ISR) fires at 200 Hz and reads all six raw axes via direct register access (bypassing the DMP), feeding the resultant-magnitude computation directly with no intermediate sensor-fusion filtering stage.

5.3.2 Sliding Window Buffer

A circular ring buffer stores the most recent samples of all six channels ($a_x, a_y, a_z, \omega_x, \omega_y, \omega_z$) in SRAM, continuously overwritten. When Layer 1 fires a `PRE_FALL` flag, the gate-triggered window is copied to a local array, normalized using StandardScaler parameters (mean and standard deviation per channel) baked into the firmware as constant arrays at training time, and passed to the TFLite Micro interpreter.

5.3.3 Layer 1 - Threshold Pre-filter

The resultant acceleration magnitude is computed every ISR tick:

$$|a| = \sqrt{a_x^2 + a_y^2 + a_z^2} \tag{5.1}$$

The `PRE_FALL` flag is set when $|a|$ exceeds 2.5 g for more than 300 ms; the queue is cleared if no new peak acceleration occurs within a 10 s timeout. These threshold values are currently fixed design parameters, pending empirical calibration against self-collected fall simulations, and are adjustable via the Strategy Pattern sensitivity profiles (Section 5.4.2).

5.3.4 Layer 2 - TFLite Micro CNN Inference

5.4 Algorithm

The following steps describe the workflow of the SPARK fall detection system:

1. START
2. Initialize all hardware components:
    - MPU6050 IMU (6-axis accelerometer + gyroscope)
    - ESP32-S3 microcontroller
    - Bluetooth Low Energy (BLE) radio
    - Laptop gateway (sole BLE receiver and sole SHAP/PDF compute)
3. Calibrate the sensor:
    - Read raw accelerometer and gyroscope values at rest.
    - Compute sensor bias/offset for calibration.
    - Store calibration constants for runtime correction.
4. Continuous sensor sampling loop:
    - Sample MPU6050 at 200 Hz over I^2C via direct register access.
    - Push the raw sample into a circular sliding window buffer, continuously overwritten.

5. Fall detection loop:

   (a) Compute resultant acceleration magnitude $|a|$ on every ISR tick.

   (b) Apply Layer 1 threshold check: $|a| > 2.5\,\text{g}$ sustained for $>300\,\text{ms}$ latches `PRE_FALL`.

   (c) If threshold is not exceeded: discard, return to Step 4.

   (d) If threshold is exceeded: copy the latched window, normalize, pass to Layer 2 (1D CNN classifier).

   (e) CNN outputs $P(\text{fall})$ and $P(\text{ADL})$ via softmax.

   (f) If $P(\text{fall}) > 0.85$: mark event as `CONFIRMED_FALL`.

   (g) Else: mark as ADL (Activity of Daily Living) and return to Step 4.

   (h) Log detection result with timestamp on-device.

6. Event publishing and transmission:

   - Serialize the `CONFIRMED_FALL` event as JSON per the locked wire format (Section 5.4.1).
   - Transmit over BLE directly to the laptop gateway — no broker or intermediate network hop.

7. Gateway event processing:

   - The gateway's BLE receiver ingests the incoming event, running locally on the laptop.
   - Run SHAP attribution on the six peak IMU features to identify which channels drove the CNN's decision.

8. Incident report generation:

   - Compile PDF report containing: timestamp, CNN confidence score, SHAP attribution chart, and signature block.
   - Save the report locally on the gateway, readable by the phone display client over the local network.

9. Return to Step 4 and continue continuous monitoring.

10. END (on system shutdown or power-off)

5.4.1 BLE Event Transmission

On `CONFIRMED_FALL`, the ESP32-S3 serializes the event as a JSON payload per the locked wire format (`event_id`, `device_id`, `firmware_version`, `timestamp_ms`, `confidence`, and six `peak_features`) and transmits it over its integrated Bluetooth 5 (LE) radio directly to the laptop gateway. The payload is well under a single BLE packet at typical negotiated MTU, requiring no chunking and no broker or intermediate network hop.

5.4.2 Sensitivity Strategy Pattern

Three sensitivity profiles are implemented as configurable parameter sets, adjustable without a firmware reflash:

- **HighSensitivity** (elderly, bed-ridden): Layer 1 threshold $|a|_{\text{spike}} = 2.0\,\text{g}$, rest duration 200 ms; Layer 2 threshold $P(\text{FALL}) > 0.75$.
- **Standard** (default): $|a|_{\text{spike}} = 2.5\,\text{g}$, rest duration 300 ms; Layer 2 threshold $P(\text{FALL}) > 0.85$.
- **SportMode** (active ADL): $|a|_{\text{spike}} = 3.0\,\text{g}$, rest duration 400 ms; Layer 2 threshold $P(\text{FALL}) > 0.90$.

5.5 Machine Learning Pipeline

5.5.1 Dataset Preparation

Training data is sourced from two datasets:

- **SisFall Dataset** [5]: 15 fall types + 19 ADL types from 38 subjects, 4,506 files at 200 Hz. All fall types used as positive class; all ADL types used as negative class. Data preparation implemented in `training/data_prep/prepare_sisfall.py` (v23, verified end-to-end).
- **Self-collected KEC Dataset** (secondary, protocol TBD): Controlled fall simulations and ADLs recorded with the SPARK node at 200 Hz. Target: ≥500 fall windows + ≥2000 ADL windows. This dataset will be published to GitHub as "KEC Wearable Fall Dataset in Nepal Context."

All windows are 3 seconds at 200 Hz (200 samples/window). Per-channel StandardScaler normalization is applied; scaler parameters (mean, std per channel) are extracted and baked into the firmware as constant arrays.

5.5.2 Classical ML Baseline

Prior to CNN training, engineered features (mean, std, min, max, range, RMS, SMA, peak resultant acceleration per channel) are used to train Random Forest and XGBoost baselines via `GridSearchCV` with `GroupKFold` subject-grouped splitting, implemented in `training/notebooks/SPARK_SisFall_ML_Pipeline.ipynb`. This provides an interpretability baseline ahead of the CNN.

5.5.3 1D CNN Training

The 1D CNN (Layer 2) is trained in TensorFlow/Keras on SisFall, then fine-tuned on the self-collected KEC dataset once the collection protocol is finalized. Training protocol: Adam optimizer (lr = 1e-3), batch size 64, 50 epochs with early stopping (patience = 10 on validation loss), 80/10/10 train/validation/test split stratified by subject — no data leakage between splits. Class weighting is applied to address the natural imbalance between fall events and ADL windows.

Evaluation metrics: Sensitivity (Recall for FALL class), Specificity (Recall for NON_FALL class), F1-score, and AUC-ROC on the held-out test set. Target: Sensitivity $\geq$ 90%, Specificity $\geq$ 90%. Figure 5.3 illustrates the CNN layer sequence and the progressive compression of the time axis.

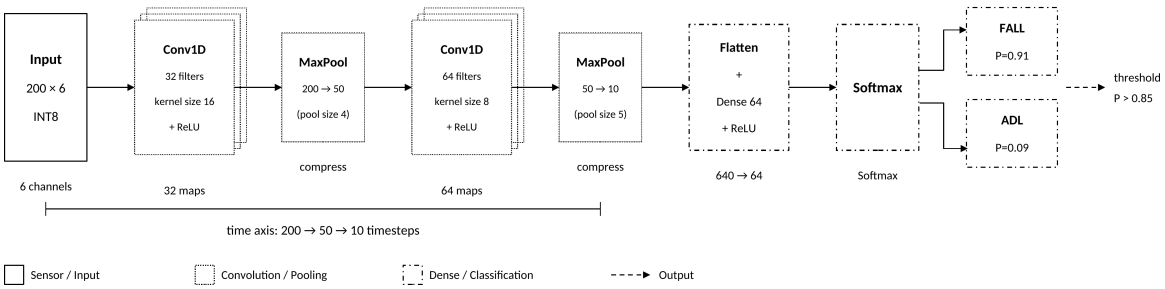


Figure 5.3: SPARK 1D CNN architecture: two Conv1D+MaxPool blocks compress the 200-sample IMU window to a 12-step feature map, which is flattened and classified by a Dense+Softmax head. INT8 quantized for TFLite Micro deployment.

5.5.4 INT8 Quantization and Firmware Integration

Post-training INT8 quantization is applied via the TFLite Converter with a representative calibration dataset (200 randomly sampled training windows). The quantized TFLite model is exported as a C byte array using the `xxd -i` command and included in the PlatformIO build as a header file. Model size target: $\leq$ 120 KB flash.

5.6 Gateway Software Implementation

5.6.1 BLE Receiver

The gateway's BLE receiver runs locally on the laptop (Acer Swift Go 16, sole BLE receiver and sole compute per §2.1). It receives each `CONFIRMED_FALL` JSON payload directly over BLE, parses it against the locked wire format (`wire_format.py`), and invokes the SHAP attribution pipeline in-process — no broker, no REST layer, and no persistent database; events are processed and reported locally, with the completed report saved to disk.

5.6.2 SHAP Attribution Pipeline

For each confirmed fall event, the gateway extracts the six peak IMU channel values (three accelerometer axes a_x, a_y, a_z and three gyroscope axes $\omega_x, \omega_y, \omega_z$, received over BLE as `peak_features`) and computes SHAP attributions using the `shap` library against the trained CNN (loaded as a TFLite interpreter on the gateway). The top contributing feature (e.g., $a_{z,\text{peak}}$, contribution 42%) and the full 6-element SHAP vector are written directly into the auto-generated PDF report — computed locally on the laptop with no cloud round-trip.

5.6.3 PDF Incident Report

ReportLab generates a one-page PDF clinical incident report per fall event, containing: institution header (KEC), event timestamp, Layer 2 CNN confidence score, SHAP top-contributing feature, full SHAP vector bar chart, and a signature block for care staff acknowledgement. The report format is modelled on the clinical incident reporting structure recommended by WHO [1] for fall event documentation. The completed report is saved locally on the gateway and read by the phone display client over the local network — no push notification or alert channel exists in this design.

5.7 Testing and Evaluation Plan

5.7.1 Unit Testing

Each subsystem is validated independently before integration: (1) MPU6050 ISR correctness verified by comparing 200 Hz output against known waveforms from a signal generator; (2) Layer 1 threshold logic verified by injecting synthetic $|a|$ sequences with known fall/non-fall labels; (3) Layer 2 TFLite Micro inference verified by comparing ESP32-S3 INT8 outputs against Python FP32 reference on 50 held-out windows; (4) BLE payload verified against the locked wire format via `test_wire_format.cpp` and a BLE sniffer capture.

5.7.2 End-to-End Latency Measurement

A GPIO pin on the ESP32-S3 is toggled at Layer 1 `PRE_FALL` and at `CONFIRMED_FALL` BLE transmit, captured on an oscilloscope to measure on-device pipeline latency. Gateway-side report completion time is recorded from BLE receipt to PDF write. Target: on-device $\leq$ 200 ms.

5.7.3 ADL False-Positive Battery

One hundred ADL sequences (walk, run, stair-climb, sit-to-stand, bend, drop object, vigorous arm swing) are performed wearing the node. The false-positive rate at Standard sensitivity profile is measured and documented. Target: false-positive rate $\leq$ 10% on the ADL battery.

5.7.4 CNN Performance Evaluation

The held-out test set (10% of combined dataset, stratified by subject) is used to compute Sensitivity, Specificity, F1-score, AUC-ROC, and confusion matrix. Results are reported for both the FP32 Keras model and the INT8 TFLite Micro model to quantify quantization accuracy loss.

Chapter 6: Expected Output

This chapter describes the deliverables, performance targets, and project schedule for SPARK. The system is developed in three sequential phases: proposal and planning (June–July 2026), skill development and dataset preparation (July–August 2026), and core build, integration, and evaluation (September 2026 – March 2027).

6.1 Deliverables

SPARK produces the following tangible deliverables by the March 2027 demonstration deadline:

6.1.1 Hardware Deliverables

1. **Functional wearable node** : ESP32-S3 with MPU6050 6-axis IMU, 1100 mAh Li-ion/LiPo battery, TP4056-class charge/protection circuit, mounted top-of-wrist/dorsal in a wrist-worn enclosure 3D-printed in TPU at KEC Makerspace, secured with the enclosure's built-in Velcro closure tab plus a separate Velcro wrist strap, worn over a compression arm sleeve base layer.

2. **Gateway** : Laptop (Acer Swift Go 16, already owned) as the sole BLE receiver and sole SHAP/PDF compute — no dedicated single-board computer required.

3. **Phone display client** : Read-only companion view (user's own phone, no additional hardware) that reads completed reports from the laptop gateway over the local network.

4. **Power estimate report** : Documented active current draw range (110–140 mA, published-figure estimate for BLE + 200 Hz IMU polling, not yet hardware-measured — no power telemetry sensor exists in this design) and resulting endurance estimate (7.9–10 hours from the 1100 mAh cell), with on-hardware profiling planned for thesis Chapter 5 (Implementation).

6.1.2 Software Deliverables

1. **ESP32-S3 firmware** (PlatformIO/Arduino C++): MPU6050 ISR at 200 Hz via direct register access, sliding window ring buffer, Layer 1 threshold pre-filter, Layer 2 TFLite Micro INT8 CNN inference, and BLE JSON event transmission per the locked wire format.

2. **Trained and quantized 1D CNN model** — INT8 TFLite model file ($\leq$ 120 KB)

achieving Sensitivity ≥ 90% and Specificity ≥ 90% on the held-out test set, integrated as a C header array in the PlatformIO build.

3. **Gateway software** (Python): BLE receiver parsing the locked wire format, SHAP attribution pipeline (per-event, no cloud round-trip, no persistent database), and ReportLab PDF incident report generator — all running locally on the laptop.

4. **Phone display client software**: Read-only report viewer that pulls completed reports from the laptop over the local network; no SHAP computation or PDF generation on-device.

5. **Self-collected KEC Wearable Fall Dataset**: Minimum 500 labelled fall event windows and 2000 ADL windows recorded at KEC campus using the SPARK node, with subject metadata, published to GitHub under an open licence as "KEC Wearable Fall Dataset in Nepal Context."

6. **GitHub repository** : Complete open-source repository containing firmware, gateway software, trained model, dataset, KiCad schematic (breadboard-level), and a reproducible README enabling replication of the full system.

Gantt chart

Table 6.1 maps all major tasks to a 32-week schedule spanning September 2026 through March 2027 (Phase 3 only). Weeks 1 — 8 correspond to September — October 2026; weeks 9 — 16 to November — December 2026; weeks 17 — 24 to January — February 2027; weeks 25 — 32 to March 2027. Filled cells (•) indicate scheduled activity.

Table 6.1: Project Gantt Chart (Phase 3: September 2026 — March 2027)

Task	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	W16
Self-collected dataset collection	•	•	•	•	•	•										
CNN training (UCI+SisFall baseline)	•	•	•	•												
CNN fine-tuning (KEC dataset)					•	•	•	•								
INT8 quantization + ESP32 flash							•	•								
ESP32 firmware (Layer 1 + Layer 2)	•	•	•	•	•	•	•	•								
Hardware node assembly + enclosure	•	•	•	•												
Laptop gateway BLE receiver	•	•	•	•												
SHAP attribution pipeline					•	•	•	•								
PDF report auto-generation pipeline						•	•	•	•							
Phone display client								•	•							
End-to-end integration + dry run								•	•							
ADL false-positive battery test									•	•						
Sensitivity profile tuning									•	•						
KEC dataset published to GitHub											•					
Thesis Chapters 1 — 4 draft											•	•	•			
Full system integration test												•	•			
F1/AUC evaluation on test set												•	•			
Thesis Chapters 5 — 6 draft													•	•		
Conference paper draft														•	•	
Final demo preparation															•	•
Thesis submission + defence																•

Note: W1 — W4 = September 2026; W5 — W8 = October 2026; W9 — W12 = November — December 2026; W13 — W16 = January — March 2027.

References

[1] World Health Organization, *WHO Global Report on Falls Prevention in Older Age*. Geneva, Switzerland: World Health Organization, 2007.

[2] Central Bureau of Statistics, Nepal, *National Population and Housing Census 2021: Population Projection Report*. Kathmandu, Nepal: National Statistics Office, Government of Nepal, 2021.

[3] M. Kangas, A. Konttila, P. Lindgren, I. Winblad, and T. Jämsä, "Comparison of low-complexity fall detection algorithms for body attached accelerometers," *Gait & Posture*, vol. 28, no. 2, pp. 285–291, 2008.

[4] A. K. Bourke, J. V. O'Brien, and G. M. Lyons, "Evaluation of a threshold-based tri-axial accelerometer fall detection algorithm," *Gait & Posture*, vol. 26, no. 2, pp. 194–199, 2008.

[5] A. Sucerquia, J. D. López, and J. F. Vargas-Bonilla, "SisFall: A fall and movement dataset," *Sensors*, vol. 17, no. 1, p. 198, 2017.

[6] O. Baños *et al.*, "mHealthDroid: a novel framework for agile development of mobile health applications," in *Proc. 6th Int. Work-Conf. Ambient Assisted Living and Daily Activities (IWAAL)*, 2014, pp. 91–98.

[7] Y.-H. Nho, J. G. Lim, and D.-S. Kwon, "Deep convolutional neural network for fall detection using inertial sensor data from wearable devices," *IEEE Sensors J.*, vol. 21, no. 22, pp. 26305–26315, 2021.

[8] Y. Wang, M. Chen, X. Wang, R. H. M. Chan, and W. J. Li, "LSTM-based fall detection system using wearable sensors," *IEEE Sensors J.*, vol. 20, no. 11, pp. 6107–6116, 2020.

[9] R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proc. Machine Learning and Systems (MLSys)*, vol. 3, 2021, pp. 800–811.

[10] A. Kumar, V. Sharma, and R. Singh, "Microcontroller-based fall detection for elderly monitoring using a threshold-gated classifier on Arduino Nano 33 BLE Sense," *Int. J. Embedded Syst. Appl.*, vol. 12, no. 1, pp. 1–12, 2022.

[11] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017,

pp. 4765–4774.

[12] G. Yao *et al.*, "Interpretation of electrocardiogram heartbeat by CNN and GRU," *Computational and Mathematical Methods in Medicine*, vol. 2021, 2021.

APPENDIX

A.1 Self-Collected Dataset Protocol

The SPARK self-collected dataset is recorded at KEC campus using the SPARK wearable node (ESP32 + MPU6050 at 100 Hz). All four team members plus recruited volunteers act as subjects. The protocol defines four fall types and five ADL types as listed below.

Fall types (positive class):

- Forward trip fall — simulated stumble onto a crash mat
- Lateral fall (left) — sideways fall onto crash mat
- Lateral fall (right) — sideways fall onto crash mat
- Sitting-to-floor fall — controlled seat loss from a chair

ADL types (negative class):

- Walking (normal pace, indoor)
- Running (moderate pace, corridor)
- Stair climbing (ascending and descending, KEC stairwell)
- Sit-to-stand transition (chair, rapid)
- Bending to pick up an object

Each fall is repeated a minimum of 5 times per subject. ADL sequences are recorded in 2-minute continuous blocks. The node is worn on the dominant wrist during all recordings. Raw CSV output is labelled using a post-hoc annotation script keyed to the Layer 1 flag timestamp.

A.2 ESP32 Hardware Connections

Wearable Node Pin Connections

Component	ESP32 Pin	Function
MPU6050 SDA	GPIO 21	I^2C Data
MPU6050 SCL	GPIO 22	I^2C Clock
MPU6050 VCC	3.3 V rail	Power
MPU6050 GND	GND	Ground
INA219 SDA	GPIO 21	I^2C Data (address 0x41)
INA219 SCL	GPIO 22	I^2C Clock
INA219 VIN+	After LDO output	Current sense (series)
TP4056 OUT+	LDO AMS1117 IN	Battery regulated output
AMS1117-3.3 OUT	ESP32 VIN	3.3 V regulated supply

A.3 PostgreSQL Schema

```
CREATE TABLE fall_events (
    event_id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id            VARCHAR(64) NOT NULL,
    timestamp          TIMESTAMPTZ NOT NULL,
    layer1_flag        BOOLEAN NOT NULL,
    layer2_confidence  FLOAT NOT NULL,
    fall_confirmed     BOOLEAN NOT NULL,
    ax_peak            FLOAT,
    ay_peak            FLOAT,
    az_peak            FLOAT,
    severity_score     INT CHECK (severity_score BETWEEN 1 AND 5),
    shap_top_feature   VARCHAR(32),
    shap_json          JSONB,
    alert_sent         BOOLEAN DEFAULT FALSE,
    false_positive_flagged BOOLEAN DEFAULT FALSE
);
```